

Detailed Test Design and Execution

1. Selected Major Module

The selected detailed module is:

- `coupon_discount_engine`

This module is a strong final-project target because it combines:

- valid and invalid input partitions
- multiple numeric boundaries
- interacting business rules
- explicit expected results
- a compact reference implementation suitable for white-box execution

2. Requirement Basis and Scope

2.1 Formalized Requirement Summary

The detailed execution document uses the following normalized rules derived from the final requirement set and the selected reference implementation:

- only one coupon may be applied to an order
- unknown coupon codes must be rejected
- expired coupons must be rejected
- `SAVE10` requires `subtotal >= 50`
- `SAVE20` requires `subtotal >= 100`
- `SAVE20` also requires premium membership
- `SAVE20` cannot be combined with sale items
- `FREESHIP` requires `subtotal >= 30`
- no-coupon input should leave subtotal and shipping unchanged

2.2 Implementation Under Test

Item	Path
Reference implementation	<code>reference_impl/coupon_discount_engine.py</code>
Black-box tests	<code>tests/test_coupon_discount_engine_black_box.py</code>

Item	Path
White-box tests	tests/test_coupon_discount_engine_whitebox.py
Seeded mutants	reference_impl/coupon_discount_engine_mutants.py

3. Test Environment and Tooling

Tool	Purpose
pytest	execute black-box and white-box test cases
coverage.py	collect statement and branch coverage
mutant functions	demonstrate defect detection usefulness

Commands used:

```
python -m pytest tests\test_coupon_discount_engine_blackbox.py tests\test_coupon_discount_engine_whitebox.py
python -m pytest tests -q
python -m coverage run --branch -m pytest tests -q
python -m coverage report -m reference_impl\coupon_discount_engine.py
python -m coverage xml -o final_docs\execution_evidence\coupon_discount_engine_coverage.xml
python -m coverage xml -o final_docs\execution_evidence\coupon_discount_engine_branch_coverage.xml
```

4. Black-Box Test Design

4.1 Equivalence Partitioning

Partition ID	Partition type	Representative condition	Designed test(s)	Expected result
EP1	valid	no coupon provided	BB01	accept and keep subtotal/shipping unchanged
EP2	invalid	more than one coupon provided	BB02	reject with one-coupon-only reason
EP3	invalid	unknown coupon code	BB03	reject with unknown-coupon reason
EP4	invalid	expired coupon	BB04	reject with expired-coupon reason
EP5	invalid	premium-only coupon used by non-premium customer	BB07	reject with membership reason

Partition ID	Partition type	Representative condition	Designed test(s)	Expected result
EP6	invalid	restricted coupon combined with sale items	BB08	reject with sale-item restriction reason
EP7	valid	SAVE20 with all preconditions satisfied	BB09	accept and apply 20% discount

4.2 Boundary Value Analysis

Boundary ID	Threshold	Below	On boundary	Above / valid representative	Designed test(s)
B1	SAVE10 subtotal 50	49	50	60	BB05, BB06, WB05
B2	SAVE20 subtotal 100	99	100	120	WB03, BB09
B3	FREESHIP subtotal 30	29	30	80 with no coupon	WB04, BB10, BB01

4.3 Decision Table

Rule	Coupon	Threshold met	Premium member	Sale items present	Expired	Expected outcome
D1	none	N/A	N/A	N/A	N/A	accept, no change
D2	SAVE10	no	N/A	N/A	no	reject
D3	SAVE10	yes	N/A	N/A	no	accept, 10% discount
D4	SAVE20	yes	no	no	no	reject
D5	SAVE20	yes	yes	yes	no	reject
D6	SAVE20	yes	yes	no	no	accept, 20% discount
D7	FREESHIP	no	N/A	N/A	no	reject
D8	FREESHIP	yes	N/A	N/A	no	accept, shipping becomes zero

4.4 Executable Black-Box Cases

Test ID	pytest function	Main technique	Covered rule / purpose
BB01	test_no_coupon_keeps_order_values	EP	no-coupon valid partition
BB02	test_multiple_coupons_are_rejected	EP	one-coupon-only rejection
BB03	test_unknown_coupon_is_rejected	EP	unknown coupon rejection
BB04	test_expired_coupon_is_rejected	EP	expired coupon rejection
BB05	test_save10_boundary_below_threshold_is_rejected	BVA	SAVE10 just below threshold
BB06	test_save10_boundary_on_threshold_is_accepted	BVA	SAVE10 exact threshold acceptance
BB07	test_save20_requires_premium_membership	EP / decision table	premium requirement
BB08	test_save20_with_sale_items_is_rejected	EP / decision table	sale-item restriction
BB09	test_save20_valid_case_applies_discount	decision table	valid SAVE20 acceptance
BB10	test_freeship_threshold_on_boundary_sets_shipping_to_zero	BVA	FREESHIP exact threshold acceptance

5. White-Box Test Design

5.1 White-Box Objectives

The white-box design targets:

- input guard branches for negative values
- all coupon dispatch branches
- rejection branches for each invalid condition
- acceptance branches for SAVE10, SAVE20, and FREESHIP
- normalization behavior for mixed-case and whitespace coupon input

5.2 Branch-to-Test Mapping

White-box obligation	Designed test(s)
subtotal < 0 guard raises ValueError	WB01
shipping_fee < 0 guard raises ValueError	WB02
SAVE20 threshold-reject branch	WB03
FREESHIP threshold-reject branch	WB04
coupon normalization branch	WB05
SAVE10 accept path	BB06, WB05
SAVE20 accept path	BB09
FREESHIP accept path	BB10

5.3 Executable White-Box Cases

Test ID	pytest function	Covered white-box purpose
WB01	test_negative_subtotal_raises_value_error	negative subtotal guard
WB02	test_negative_shipping_fee_raises_value_error	negative shipping fee guard
WB03	test_save20_below_threshold_keeps_original_values	SAVE20 threshold rejection branch
WB04	test_freeship_below_threshold_is_rejected	FREESHIP rejection branch
WB05	test_coupon_normalization_accepts_mixed_case_and_spacing	normalization and SAVE10 valid path

6. Execution Results

Detailed module execution evidence

coupon_discount_engine combines black-box design, white-box execution, and mutation usefulness evidence.

Executable tests 15 module tests 27 repo tests	Coverage 100% statement 100% branch	Mutation result 4 / 4 mutants killed
---	--	---

Evidence sources: tests/test_coupon_discount_engine_blackbox.py, tests/test_coupon_discount_engine_whitebox.py, coverage XML, and coupon_discount_engine_mutation_demo.md.

Detailed Module Execution Evidence

6.1 Summary of Observed Results

Item	Observed result
Module-specific executable tests	15 passed
Repository regression suite at report-preparation time	27 passed
Statement coverage on the reference module	100%
Branch coverage on the reference module	100%
Mutation result	4 / 4 mutants killed

6.2 Coverage Summary

Name	Stmts	Miss	Branch	BrPart	Cover
reference_impl\coupon_discount_engine.py	51	0	26	0	100%
TOTAL	51	0	26	0	100%

Coverage interpretation:

- black-box design is strong enough to exercise the functional rule structure
- white-box design closes the remaining branch obligations
- no branch in the selected reference implementation remains unexecuted

6.3 Result Analysis

The detailed execution result is strong for three reasons.

First, the black-box suite is not superficial. It covers valid partitions, invalid partitions, multiple thresholds, and interacting rule conditions such as premium membership plus sale-item restrictions.

Second, the white-box suite is not decorative. It exercises the explicit negative-input guards and rejection/acceptance branches that are easy to overlook in a purely requirement-level discussion.

Third, the combined suite is compact. With only 15 module-focused cases, it achieves complete statement and branch coverage on the selected implementation, which is a good tradeoff between completeness and maintainability for a coursework final project.

7. Mutation-Based Usefulness Demonstration

The final project also includes defect-seeded usefulness evidence. Four representative mutants were created:

- a mutant that allows multiple coupons
- a mutant that rejects SAVE10 exactly at subtotal 50
- a mutant that ignores the SAVE20 sale-item restriction
- a mutant that rejects FREESHIP exactly at subtotal 30

Observed outcome:

- 4 / 4 mutants were killed

Mutation-to-test mapping:

Mutant	Killed by
multiple_coupons_allowed	BB02, BB08
save10_boundary_bug	BB06
save20_sale_item_bug	BB08
freeship_boundary_bug	BB10

This is important because it shows that the detailed suite is not only structurally complete. It is also effective at detecting realistic logic defects that correspond to the selected business rules and boundaries.

8. Evidence Paths

Primary evidence files:

- final_docs/execution_evidence/coupon_discount_engine_execution_summary.md
- final_docs/execution_evidence/coupon_discount_engine_coverage.xml
- final_docs/execution_evidence/coupon_discount_engine_branch_coverage.xml
- final_docs/execution_evidence/coupon_discount_engine_mutation_demo.md

9. Conclusion

The coupon_discount_engine module satisfies the detailed-design requirement of the final project at a strong level. The module is supported by multiple black-box techniques, executable white-box tests, complete statement and branch coverage, and a successful mutation demonstration. This makes it a credible detailed anchor for the overall ARG-Test submission rather than a merely illustrative example.